ECE425: Microprocessor Systems

Programming Part2: GPIO Driver Development

Spring 2026

Myung (Michael) Cho

Review of Previous Lecture

Today’s Lecture:

ARM Cortex-M4 Based Microcontroller : GPIO Driver Development

What is a driver?

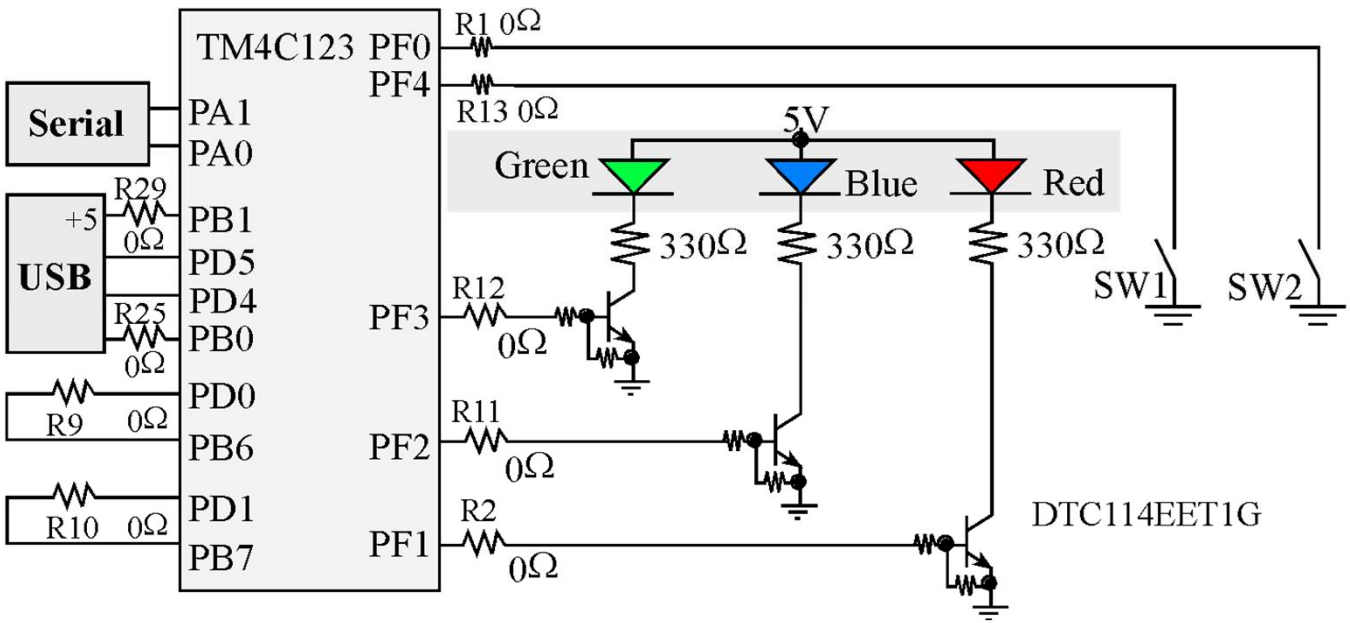
- In embedded systems, a driver is a software component that controls and manages specific hardware peripherals or devices.
  - Provides a standardized way for the application software to interact with the hardware.
  - Abstracts low-level details such as direct register manipulation (i.e. manually changing bits) and provides a high-level interface for the user.

- Throughout the semester, you will write a driver for each peripheral (e.g. GPIO, Timers, PWM, etc.) that we will cover.
  - You will have a collection of drivers to use for your final project.

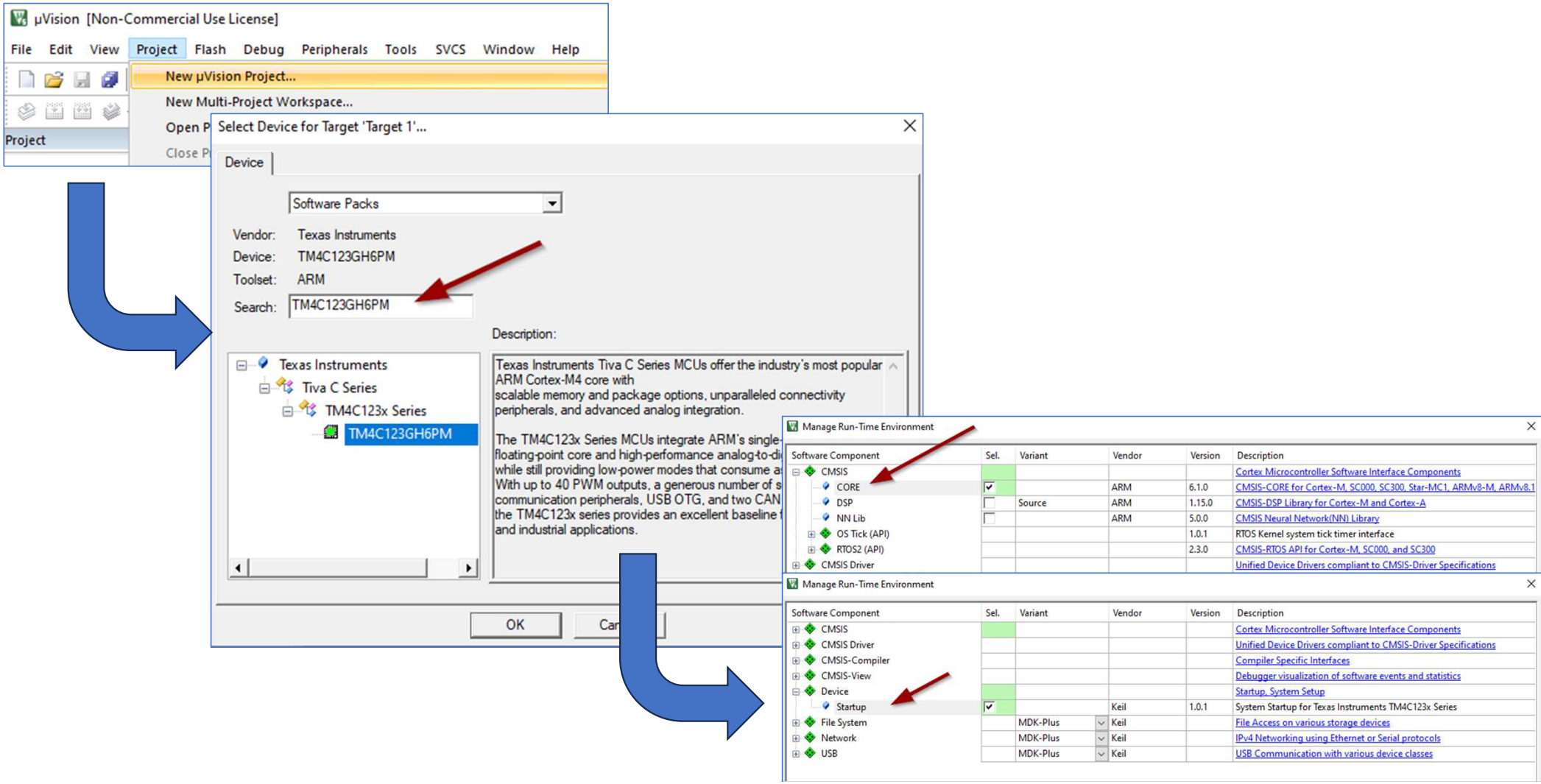
- We will go over an example that demonstrates how to write a GPIO driver for the RGB LED and the push buttons.

LaunchPad Switches and LEDs

- The switches on the LaunchPad
  - Negative logic
  - Require internal pull-up (set bits in PUR).
- The PF3-1 LEDs are positive logic.



(Appendix) Create a Project in Keil Software



(Example) Blink LED Implementation- LEDs on GPIO

main.c*

```c
/*
  Blinking LEDs through Port F
*/

#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{
    // GPIO Initialization as Output(Port F)
    SYSCTL->RCGCGPIO |= 0x20;
    GPIOF->DIR |= 0x0E;
    GPIOF->AFSEL &= ~(0x0E);
    GPIOF->DEN |= 0x0E;
    GPIOF->DATA &= ~(0x0E); // initially all 0 (turn off)

    //Blink LED
    for( ; ; )
    {
        GPIOF->DATA = 0x02; // Turn on PF1 (Red)
        delayMs(1);
        GPIOF->DATA = 0x04; // Turn on PF2 (Blue)
        delayMs(1);
        GPIOF->DATA = 0x08; // Turn on PF3 (Green)
        delayMs(1);
    }
}

// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <3000000; j++)
        {
          //do nothing
        }
    }
}
```

main() function

(Example) Blink LED Implementation- LEDs on GPIO

main.c*

```c
/*
  Blinking LEDs through Port F
*/

#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{
    // GPIO Initialization as Output(Port F)
    SYSCTL->RCGCGPIO |= 0x20;
    GPIOF->DIR |= 0x0E;
    GPIOF->AFSEL &= ~(0x0E);
    GPIOF->DEN |= 0x0E;
    GPIOF->DATA &= ~(0x0E); // initially all 0 (turn off)

    //Blink LED
    for( ; ; )
    {
        GPIOF->DATA = 0x02; // Turn on PF1 (Red)
        delayMs(1);
        GPIOF->DATA = 0x04; // Turn on PF2 (Blue)
        delayMs(1);
        GPIOF->DATA = 0x08; // Turn on PF3 (Green)
        delayMs(1);
    }
}

// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <3000000; j++)
        {
          //do nothing
        }
    }
}
```

Local delay() function

(Example) Blink LED Implementation- LEDs on GPIO

main.c*

```c
/*
  Blinking LEDs through Port F
*/

#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{
    // GPIO Initialization as Output(Port F)
    SYSCTL->RCGCGPIO |= 0x20;
    GPIOF->DIR |= 0x0E;
    GPIOF->AFSEL &= ~(0x0E);
    GPIOF->DEN |= 0x0E;
    GPIOF->DATA &= ~(0x0E); // initially all 0 (turn off)

    //Blink LED
    for( ; ; )
    {
        GPIOF->DATA = 0x02; // Turn on PF1 (Red)
        delayMs(1);
        GPIOF->DATA = 0x04; // Turn on PF2 (Blue)
        delayMs(1);
        GPIOF->DATA = 0x08; // Turn on PF3 (Green)
        delayMs(1);
    }
}

// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <3000000; j++)
        {
          //do nothing
        }
    }
}
```

Initialization
(Setup registers)

(Example) Blink LED Implementation- LEDs on GPIO

main.c*

```c
/*
  Blinking LEDs through Port F
*/

#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{
    // GPIO Initialization as Output(Port F)
    SYSCTL->RCGCGPIO |= 0x20;
    GPIOF->DIR |= 0x0E;
    GPIOF->AFSEL &= ~(0x0E);
    GPIOF->DEN |= 0x0E;
    GPIOF->DATA &= ~(0x0E); // initially all 0 (turn off)

    //Blink LED
    for( ; ; )
    {
        GPIOF->DATA = 0x02; // Turn on PF1 (Red)
        delayMs(1);
        GPIOF->DATA = 0x04; // Turn on PF2 (Blue)
        delayMs(1);
        GPIOF->DATA = 0x08; // Turn on PF3 (Green)
        delayMs(1);
    }
}

// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <3000000; j++)
        {
          //do nothing
        }
    }
}
```

infinite loop

Blinking LED
(by changing data register)

GPIO Initialization as Output

- Write an initialization function called RGB_LED_Init for the RGB LED, which uses the following pins:
  - PF1, PF2, and PF3
  - Refer to the user's guide for the LaunchPad schematic.

```
9
10  void RGB_LED_Init(void)
11  {
12
13  }
14
```

GPIO Initialization as Output

- Write an initialization function called RGB_LED_Init for the RGB LED, which uses the following pins:
  - PF1, PF2, and PF3
  - Refer to the user's guide for the LaunchPad schematic.

(Question) What bit mask must be used for the RGB LED?

```
void RGB_LED_Init(void)
{

}
```

GPIO Initialization as Output

- Write an initialization function called RGB_LED_Init for the RGB LED, which uses the following pins:
  - PF1, PF2, and PF3
  - Refer to the user's guide for the LaunchPad schematic.

(Question) What bit mask must be used for the RGB LED?

➔ Since we're interested in using Bit 3 to 1, we will be using a bit mask of 0x0E (0000_1110b).

```
9
10  void RGB_LED_Init(void)
11  {
12
13  }
14
```

GPIO Initialization as Output - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register. (Verify the bit in the datasheet!)

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

GPIO Initialization as Output - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register. (Verify the bit in the datasheet!)

➔ Need to write a value of 0x20 (0010_0000b) in the RCGCGPIO register since we're setting Bit 5 to 1.

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

GPIO Initialization as Output - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register. (Verify the bit in the datasheet!)

➔ Need to write a value of 0x20 (0010_0000b) in the RCGCGPIO register since we're setting Bit 5 to 1.

➔ Perform an OR operation with the current value of the RCGCGPIO register and the bit mask (0x20)

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

GPIO Initialization as Output - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register. (Verify the bit in the datasheet!)

➔ Need to write a value of 0x20 (0010_0000b) in the RCGCGPIO register since we're setting Bit 5 to 1.

➔ Perform an OR operation with the current value of the RCGCGPIO register and the bit mask (0x20)

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

```
9
10  void RGB_LED_Init(void)
11  {
12      SYSCTL->RCGCGPIO = SYSCTL->RCGCGPIO | 0x20;
13
14  }
15
```

(Or "SYSCTL -> RCGCGPIO |= 0x20;")

GPIO Initialization as Output - GPIODIR

- Set the direction of the PF1, PF2, and PF3 pins as output by setting the bits in the DIR field of the GPIODIR register.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DIR							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DIR	RW	0x00	GPIO Data Direction Value Description 0 Corresponding pin is an input. 1 Corresponding pins is an output.

GPIO Initialization as Output - GPIODIR

- Set the direction of the PF1, PF2, and PF3 pins as output by setting the bits in the DIR field of the GPIODIR register.

➔ Perform an OR operation with the current value of the GPIODIR register and the bit mask (0x0E) to set specific bits.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DIR							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DIR	RW	0x00	GPIO Data Direction Value Description 0 Corresponding pin is an input. 1 Corresponding pins is an output.

GPIO Initialization as Output - GPIODIR

- Set the direction of the PF1, PF2, and PF3 pins as output by setting the bits in the DIR field of the GPIODIR register.

➔ Perform an OR operation with the current value of the GPIODIR register and the bit mask (0x0E) to set specific bits.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DIR							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DIR	RW	0x00	GPIO Data Direction Value Description 0 Corresponding pin is an input. 1 Corresponding pins is an output.

```c

void RGB_LED_Init(void)
{
  SYSCTL->RCGCGPIO = SYSCTL->RCGCGPIO | 0x20;

  GPIOF->DIR = GPIOF->DIR | 0x0E;

}

```

GPIO Initialization as Output - GPIOAFSEL

- Configure the PF1, PF2, and PF3 pins to function as GPIO pins by clearing the bits of the AFSEL field in the GPIOAFSEL register.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								AFSEL							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	AFSEL	RW	-	GPIO Alternate Function Select Value Description 0 The associated pin functions as a GPIO and is controlled by the GPIO registers. 1 The associated pin functions as a peripheral signal and is controlled by the alternate hardware function. The reset value for this register is 0x0000.0000 for GPIO ports that are not listed in Table 10-1 on page 650.

GPIO Initialization as Output - GPIOAFSEL

- Configure the PF1, PF2, and PF3 pins to function as GPIO pins by clearing the bits of the AFSEL field in the GPIOAFSEL register.

➔ Perform an AND operation with the current value of the GPIOAFSEL register and the complement of the bit mask (~0x0E) to clear specific bits

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								AFSEL							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	AFSEL	RW	-	GPIO Alternate Function Select

Value	Description
0	The associated pin functions as a GPIO and is controlled by the GPIO registers.
1	The associated pin functions as a peripheral signal and is controlled by the alternate hardware function.

The reset value for this register is 0x0000.0000 for GPIO ports that are not listed in Table 10-1 on page 650.

```
9
10  void RGB_LED_Init(void)
11  {
12      SYSCTL->RCGCGPIO = SYSCTL->RCGCGPIO | 0x20;
13
14      GPIOF->DIR = GPIOF->DIR | 0x0E;
15
16      GPIOF->AFSEL = GPIOF->AFSEL & ~(0x0E);
17
18  }
19
```

GPIO Initialization as Output - GPIODEN

- Enable the digital function on the PF1, PF2, and PF3 pins by setting the bits in the DEN field of the GPIODEN register.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DEN							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DEN	RW	-	Digital Enable

Value	Description
0	The digital functions for the corresponding pin are disabled.
1	The digital functions for the corresponding pin are enabled.

The reset value for this register is 0x0000.0000 for GPIO ports that are not listed in Table 10-1 on page 650.

GPIO Initialization as Output - GPIODEN

- Enable the digital function on the PF1, PF2, and PF3 pins by setting the bits in the DEN field of the GPIODEN register.

➔ Perform an OR operation with the current value of the GPIODEN register and the bit mask (0x0E) to set specific bits

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DEN							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DEN	RW	-	Digital Enable

Value	Description
0	The digital functions for the corresponding pin are disabled.
1	The digital functions for the corresponding pin are enabled.

The reset value for this register is 0x0000.0000 for GPIO ports that are not listed in Table 10-1 on page 650.

```
9
10  void RGB_LED_Init(void)
11  {
12      SYSCTL->RCGCGPIO = SYSCTL->RCGCGPIO | 0x20;
13
14      GPIOF->DIR = GPIOF->DIR | 0x0E;
15
16      GPIOF->AFSEL = GPIOF->AFSEL & ~(0x0E);
17
18      GPIOF->DEN = GPIOF->DEN | 0x0E;
19
20  }
21
```

GPIO Initialization as Output - GPIODATA

- Ensure that the RGB LED is off after initialization by clearing the bits in the DATA field of the GPIODATA register.

➔ Perform an AND operation with the current value of the GPIODATA register and the complement of the bit mask (~0x0E) to clear specific bits

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

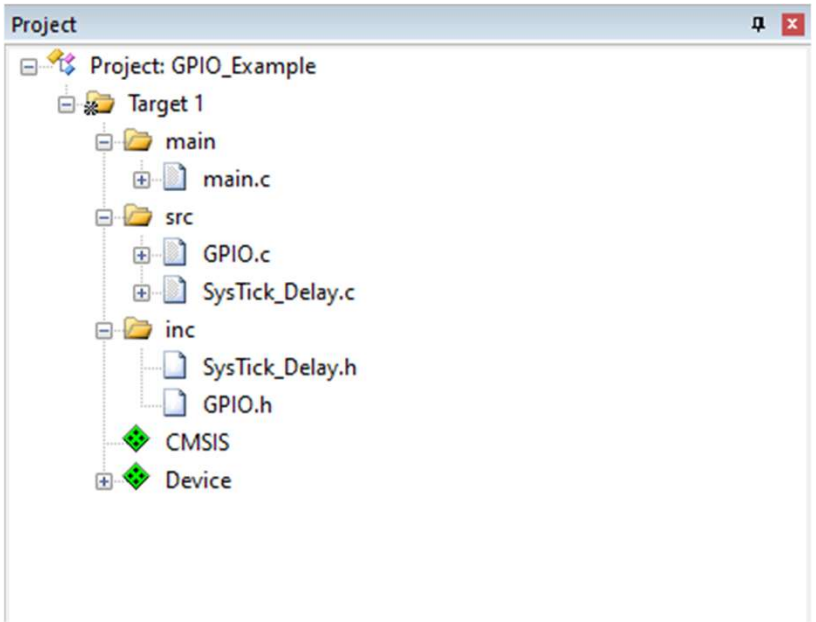
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DATA							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DATA	RW	0x00	GPIO Data This register is virtually mapped to 256 locations in the address space. To facilitate the reading and writing of data to these registers by independent drivers, the data read from and written to the registers are masked by the eight address lines [9:2]. Reads from this register return its current state. Writes to this register only affect bits that are not masked by ADDR[9:2] and are configured as outputs. See "Data Register Operation" on page 654 for examples of reads and writes.

```
9
10  void RGB_LED_Init(void)
11  {
12      SYSCTL->RCGCGPIO = SYSCTL->RCGCGPIO | 0x20;
13
14      GPIOF->DIR = GPIOF->DIR | 0x0E;
15
16      GPIOF->AFSEL = GPIOF->AFSEL & ~(0x0E);
17
18      GPIOF->DEN = GPIOF->DEN | 0x0E;
19
20      GPIOF->DATA = GPIOF->DATA &= ~0x0E;
21  }
22
```

(Appendix) Project Creation

- Create a project named GPIO_Example
- Add the following file groups
  - main
  - src
  - inc
- Create the following source files
  - main.c (main)
  - GPIO.c (src)
  - SysTick_Delay.c (src)
- Create the following header files
  - GPIO.h (inc)
  - SysTick_Delay.h (inc)



(Appendix) Project Creation

- Copy the contents of the SysTick_Delay.c and SysTick_Delay.h files from the Blinking LED project into your project.
  - The SysTick_Delay driver will be covered in the Timers lecture.

SysTick_Delay.c | SysTick_Delay.h

```c
/**
 * @file SysTick_Delay.c
 *
 * @brief Source code for the SysTick_Delay driver.
 *
 * It provides a blocking function (SysTick_Delay1ms) to create a delay with a busy-wait loop
 * and it uses the SysTick timer with a specified reload value to generate interrupts every 1 ms.
 * It uses the Peripheral Internal Oscillator (PIOSC) as the clock source.
 * The PIOSC provides 16 MHz which is then divided by 4. The timer is used for creating delays in milliseconds.
 *
 * @author Aaron Nanas
 */

#include "SysTick_Delay.h"

// Global variable used to keep track of elapsed time in microseconds
static uint32_t us_elapsed = 0;

// Global variable used to keep track of elapsed time in milliseconds
static uint32_t ms_elapsed = 0;

// Global flag used to indicate if milliseconds delay is active
static uint8_t ms_active = 0;

void SysTick_Delay_Init(void)
{
  // Set the SysTick timer reload value for 1 ms intervals
  SysTick->LOAD = (4 - 1);

  // Enable the SysTick timer and its interrupt
  // with the Peripheral Internal Oscillator (PIOSC) as the clock source
  SysTick->CTRL |= 0x03;
}
```

(Appendix) Output Initialization

- In the GPIO.h header file, include the TM4C123GH6PM.h header file.
  - This provides access to the peripheral registers.
- Declare global constant variables that will be used for setting specific colors on the RGB LED.
  - The extern keyword allows the global variable to be used anywhere in the program.
- Declare the function prototype of RGB_LED_Init.

GPIO.c GPIO.h

```
#include "TM4C123GH6PM.h"

// Constant definitions for the user LED (RGB) colors
extern const uint8_t RGB_LED_OFF;
extern const uint8_t RGB_LED_RED; // PF1
extern const uint8_t RGB_LED_BLUE; // PF2
extern const uint8_t RGB_LED_GREEN; // PF3

void RGB_LED_Init(void);
```

(Appendix) Output Initialization

- Initialize the global constant variables in the GPIO.c source file.
- These values correspond to the bits being set.
  - For example, if PF1 is set to 1, then the value written in the GPIODATA register is 0x02 (0000_0010b).

GPIO.c GPIO.h

```c

#include "GPIO.h"

// Constant definitions for the user LED (RGB) colors
const uint8_t RGB_LED_OFF     = 0x00;
const uint8_t RGB_LED_RED     = 0x02; // PF1
const uint8_t RGB_LED_BLUE    = 0x04; // PF2
const uint8_t RGB_LED_GREEN   = 0x08; // PF3

```

(Appendix) Output Initialization

- Write a function called RGB_LED_Output() that takes in a uint8_t parameter and can be used to turn on or off the RGB LED.
- Use the GPIODATA register to change the value of the PF1, PF2, and PF3 pins and perform an OR operation with the uint8_t parameter.
  - The 0xF1 bit mask indicates that all of the other bits except for Bit 1, Bit 2, and Bit 3 should be unaffected.
- Declare the function prototype in the GPIO.h header file.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DATA							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DATA	RW	0x00	GPIO Data This register is virtually mapped to 256 locations in the address space. To facilitate the reading and writing of data to these registers by independent drivers, the data read from and written to the registers are masked by the eight address lines [9:2]. Reads from this register return its current state. Writes to this register only affect bits that are not masked by ADDR[9:2] and are configured as outputs. See "Data Register Operation" on page 654 for examples of reads and writes.

```c
void RGB_LED_Output(uint8_t led_value)
{
    GPIOF->DATA = (GPIOF->DATA & 0xF1) | led_value;
}
```

(Appendix) Output Testing

- Include the following header files in the main.c source file:
  - TM4C123GH6PM.h
  - SysTick_Delay.h
  - GPIO.h

main.c | GPIO.c | GPIO.h

```c

#include "TM4C123GH6PM.h"
#include "SysTick_Delay.h"
#include "GPIO.h"

int main(void)
{

  while(1)
  {

  }
}

```

(Appendix) Output Testing

- Call the following initialization functions in the main function:
  - SysTick_Delay_Init()
  - RGB_LED_Init()
- Initialization functions will only be executed once to set up the hardware or peripheral to be used.

main.c | GPIO.c | GPIO.h

```c

#include "TM4C123GH6PM.h"
#include "SysTick_Delay.h"
#include "GPIO.h"

int main(void)
{
  SysTick_Delay_Init();

  RGB_LED_Init();

  while(1)
  {

  }
}

```

(Appendix) Output Testing

- Test the driver by displaying a green color on the RGB LED and flashing it every 500 ms.

- Before you build and flash the project, ensure that you have the correct project settings:
  - Flash > Configure Flash Tools
  - C Compiler Version: c11
  - Debugger: Stellaris ICDI

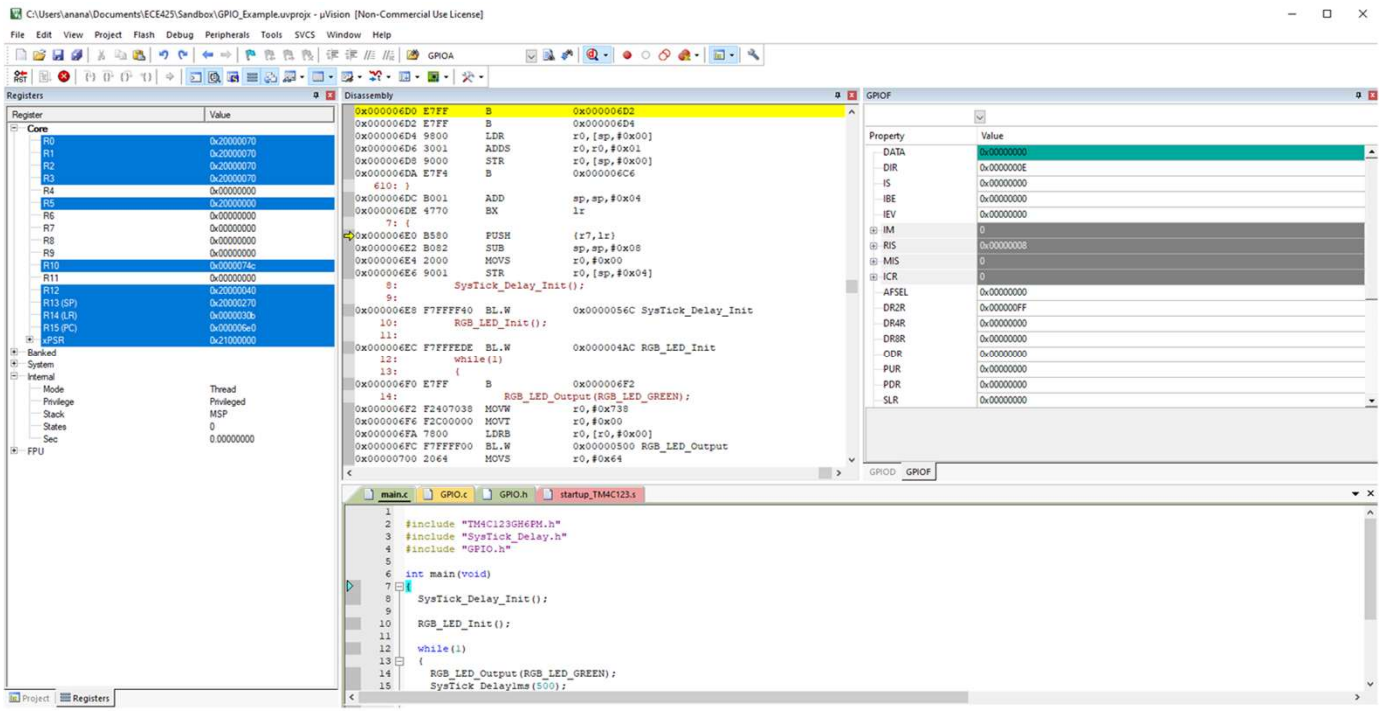
- Don't forget to press the RESET button on the TM4C123G LaunchPad after loading the program.

main.c | GPIO.c | GPIO.h

```c

#include "TM4C123GH6PM.h"
#include "SysTick_Delay.h"
#include "GPIO.h"

int main(void)
{
  SysTick_Delay_Init();

  RGB_LED_Init();

  while(1)
  {
    RGB_LED_Output(RGB_LED_GREEN);
    SysTick_Delay1ms(500);

    RGB_LED_Output(RGB_LED_OFF);
    SysTick_Delay1ms(500);
  }
}

```

(Appendix) Output Testing - Debug

- Start a debugging session and then run the program.
- Observe the GPIO F registers.
  - Peripherals > System Viewer > GPIO > GPIOF
  - Notice how the value of the DATA register changes.



GPIO Initialization as Input

- Write an initialization function called Push_Button_Init for the SW1 push button on the LaunchPad board, which uses the PF4 pin.

(Question) What bit mask must be used for the SW1 push button?

```
void Push_Button_Init(void)
{

}

```

GPIO Initialization as Input

- Write an initialization function called Push_Button_Init for the SW1 push button on the LaunchPad board, which uses the PF4 pin.

(Question) What bit mask must be used for the SW1 push button?

➔ Since we're interested in using Bit 4, we will be using a bit mask of 0x10 (0001_0000b).

```
void Push_Button_Init(void)
{

}

```

GPIO Initialization as Input - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register (Verify the bit in the datasheet!)

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

GPIO Initialization as Input - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register (Verify the bit in the datasheet!)

➔ Need to write a value of 0x20 (0010_0000b) in the RCGCGPIO register since we're setting Bit 5 to 1

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

GPIO Initialization as Input - RCGCGPIO

- Enable the clock for GPIO Port F by setting the R5 bit in the RCGCGPIO register (Verify the bit in the datasheet!)

➔ Need to write a value of 0x20 (0010_0000b) in the RCGCGPIO register since we're setting Bit 5 to 1

➔ Perform an OR operation with the current value of the RCGCGPIO register and the bit mask (0x20).

General-Purpose Input/Output Run Mode Clock Gating Control (RCGCGPIO)

Base 0x400F.E000
Offset 0x608
Type RW, reset 0x0000.0000

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved										R5	R4	R3	R2	R1	R0
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:6	reserved	RO	0	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
5	R5	RW	0	GPIO Port F Run Mode Clock Gating Control Value Description 0 GPIO Port F is disabled. 1 Enable and provide a clock to GPIO Port F in Run mode.
4	R4	RW	0	GPIO Port E Run Mode Clock Gating Control Value Description 0 GPIO Port E is disabled. 1 Enable and provide a clock to GPIO Port E in Run mode.

```
void Push_Button_Init(void)
{
    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
}
```

GPIO Initialization as Input - GPIODIR

- Set the direction of the PF4 pin as input by clearing a bit in the DIR field of the GPIODIR register.

➔ Perform an AND operation with the current value of the GPIODIR register and the complement of the bit mask (~0x10) to clear a specific bit.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								DIR							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of a reserved bit. To provide compatibility with future products, the value of a reserved bit should be preserved across a read-modify-write operation.
7:0	DIR	RW	0x00	GPIO Data Direction Value Description 0 Corresponding pin is an input. 1 Corresponding pins is an output.

```
void Push_Button_Init(void)
{
    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable

    GPIOF->DIR &= ~(0x10);      //PF0 as input
}
```

GPIO Initialization as Input - GPIOAFSEL

- Configure the PD0 pin to function as a GPIO pin by clearing a bit in the AFSEL field of the GPIOAFSEL register.

➔ Perform an AND operation with the current value of the GPIOAFSEL register and the complement of the bit mask (~0x10) to clear a specific bit.

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	reserved								AFSEL							
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the v compatibility with future products preserved across a read-modify-
7:0	AFSEL	RW	-	GPIO Alternate Function Select

Value	Description
0	The associated pin functi the GPIO registers.
1	The associated pin functi controlled by the alternat

The reset value for this re
that are not listed in Table 10-1 on page 650.

```
void Push_Button_Init(void)
{
    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable

    GPIOF->DIR &= ~(0x10);      //PF0 as input

    GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register
}
```

GPIO Initialization as Input - GPIODEN

- Enable the digital function on the PF4 pin by setting a bit in the DEN field of the GPIODEN register.

➔ Perform an OR operation with the current value of the GPIODEN register and the bit mask (0x01) to set specific bits

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	reserved															
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0					

	15	14	13	12	11	10	9	8	7	6	5
	reserved										
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on t compatibility with future prod preserved across a read-mo
7:0	DEN	RW	-	Digital Enable

Value	Description
0	The digital functions for the corresponding pin are disabled.
1	The digital functions for the corresponding pin are enabled.

The reset value for this register is 0x0000.0000 for GPIO ports that are not listed in Table 10-1 on page 650.

```
void Push_Button_Init(void)
{
    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable

    GPIOF->DIR &= ~(0x10);      //PF0 as input

    GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register

    GPIOF->DEN |= 0x10; //PF4 as digital port
}
```

GPIO Initialization as Input - GPIOPUR

- Enable the pull-down resistor on the PF4 pin by setting a bit in the PUE field of the GPIOPUR register.
  (Note: SW1 is configured to be an active-low push button.)

➔ Perform an OR operation with the current value of the GPIOPUR register and the bit mask (0x10) to set specific bits

	31	30	29	28	27	26	25	24	23	22	21	20
	reserved											
Type	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
Reset	0	0	0	0	0	0	0	0	0	0	0	0

	15	14	13	12	11	10	9	8	7	6	5	4
	reserved								PUE			
Type	RO	RO	RO	RO	RO	RO	RO	RO	RW	RW	RW	RW
Reset	0	0	0	0	0	0	0	0	-	-	-	-

Bit/Field	Name	Type	Reset	Description
31:8	reserved	RO	0x0000.00	Software should not rely on the value of compatibility with future products, the va preserved across a read-modify-write op
7:0	PUE	RW	-	Pad Weak Pull-Up Enable

Value	Description
0	The corresponding pin's weak pull-up resistor is disabled.
1	The corresponding pin's weak pull-up resistor is enabled.

Setting a bit in the **GPIOPDR** register clears the corresponding bit in the **GPIOPUR** register. The change is effective on the second clock cycle after the write if accessing GPIO via the APB memory aperture. If using AHB access, the change is effective on the next clock cycle.

The reset value for this register is 0x0000.0000 for GPIO ports that are not listed in Table 10-1 on page 650.

```c
void Push_Button_Init(void)
{
    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable

    GPIOF->DIR &= ~(0x10);      //PF0 as input

    GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register

    GPIOF->DEN |= 0x10; //PF4 as digital port

    GPIOF->PUR |= 0x10; //pull up register at PF4 enable
}
```

GPIO Initialization as Input

- Write a function called Get_Push_Button_Status() that takes no parameters and has a return type of uint8_t.
  - This function will return the status of the SW5 push button.
- Only read PF4 (Bit 4) by reading the current value of the GPIODATA register and performing an AND operation with the bit mask (0x10) and assigning the value to uint8_t button_status.

```
// get push button status
uint8_t getPushButtonStatus(void)
{
    uint8_t ButtonStatus = GPIOF->DATA & 0x10;
    return ButtonStatus;
}
```

(Appendix) GPIO Initialization as Input

- Declare the function prototypes in the GPIO.h header file.
- In the main.c source file, call the Push_Button_Init() function in the main function.

```c

void Push_Button_Init(void);

uint8_t Get_Push_Button_Status(void);


```

GPIO.h file

main.c | GPIO.c | GPIO.h

```c

#include "TM4C123GH6PM.h"
#include "SysTick_Delay.h"
#include "GPIO.h"

int main(void)
{
  SysTick_Delay_Init();

  RGB_LED_Init();

  Push_Button_Init();

  while(1)
  {

  }
}
```

Input Testing

- To test the driver for SW1, update the while loop by checking if SW1 has been pressed. In other words, check if PF4 == 0.
  - If it has been pressed, turn on the RGB LED and display a red color
  - Otherwise, turn off the RGB LED.

- Constantly checking an event in the main thread is called *polling*.

```c
/*
  Blinking LEDs through Port F with switch PF4
*/

#include "TM4C123GH6PM.h"

uint8_t getPushButtonStatus(void);
void Push_Button_Init(void);
void RGB_LED_Init(void);
void delayMs(int n);


int main(void)
{

    RGB_LED_Init();

    Push_Button_Init();

    //Blink LED
    while(1)
    {
      if (getPushButtonStatus() == 0) // switch on
        {
            GPIOF->DATA = 0x02; // Turn on PF1 (Red)
            delayMs(500);
        }
        else
        {
            GPIOF->DATA = 0x00; // LED off
            delayMs(500);
        }
    }
}
```

(Appendix) Input Testing

- Definition: Polling refers to the process where the system repeatedly checks the status of an input device, sensor, or peripheral
  - The CPU actively monitors the status in a loop and checks if the device requires attention or if it has new data available
  - Leads to inefficient CPU usage because it spends time checking devices that might not need immediate attention

```c
/*
  Blinking LEDs through Port F with switch PF4
*/

#include "TM4C123GH6PM.h"

uint8_t getPushButtonStatus(void);
void Push_Button_Init(void);
void RGB_LED_Init(void);
void delayMs(int n);


int main(void)
{

    RGB_LED_Init();

    Push_Button_Init();

    //Blink LED
    while(1)
    {
      if (getPushButtonStatus() == 0) // switch on
        {
            GPIOF->DATA = 0x02; // Turn on PF1 (Red)
            delayMs(500);
        }
        else
        {
            GPIOF->DATA = 0x00; // LED off
            delayMs(500);
        }
    }
}
```

(Example1) Input Implementation – LEDs and Switches

```c
/*
  Blinking LEDs through Port F with switch PF4
*/

#include "TM4C123GH6PM.h"

void delayMs(int n);
void RGB_LED_Init(void);
void Push_Button_Init(void);

int main(void)
{
    RGB_LED_Init();
    void RGB_LED_Init()
    Push_Button_Init();


    //Blink LED
    while(1)
    {

      uint8_t ButtonStatus = GPIOF->DATA & 0x10;

      if (ButtonStatus == 0) // switch on (default: pull up)
        {
            GPIOF->DATA = 0x02; // Turn on PF1 (Red)
            delayMs(500);
        }
        else
        {
            GPIOF->DATA = 0x00; // LED off
            delayMs(500);
        }
    }
}

```

```c
void RGB_LED_Init(void)
{
    // GPIO Output Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
    GPIOF->DIR |= 0x0E;     //PF3-1 as output PF4 as input
    GPIOF->AFSEL &= ~(0x0E); //PF3-1 as GPIO controlled by GPIO register
    GPIOF->DEN |= 0x0E; //PF4-1 as digital port
    GPIOF->DATA = 0x11; // initially all 0 output(turn off) and switch on (pull up)
}
void Push_Button_Init(void)
{
    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
    GPIOF->DIR &= ~(0x10);      //PF0 as input
    GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register
    GPIOF->DEN |= 0x10; //PF4 as digital port
    GPIOF->PUR |= 0x10; //pull up register at PF4 enable
}

// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <6000; j++)
        {
          //do nothing
        }
    }
}
```

(Example2) Input Implementation – LEDs and Switches

main() function

```c
#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{

    // GPIO Output Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
    GPIOF->DIR |= 0x0E;      //PF3-1 as output PF4 as input
    GPIOF->AFSEL &= ~(0x0E); //PF3-1 as GPIO controlled by GPIO register
    GPIOF->DEN |= 0x0E; //PF4-1 as digital port
    GPIOF->DATA = 0x11; // initially all 0 output(turn off) and switch on (pull up)

    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
    GPIOF->DIR &= ~(0x10);       //PF0 as input
    GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register
    GPIOF->DEN |= 0x10; //PF4 as digital port
    GPIOF->PUR |= 0x10; //pull up register at PF4 enable

    //Blink LED
    while(1)
    {

      uint8_t ButtonStatus = GPIOF->DATA & 0x10;

      if (ButtonStatus == 0) // switch on (default: pull up)
        {
            GPIOF->DATA = 0x02; // Turn on PF1 (Red)
            delayMs(500);
        }
        else
        {
            GPIOF->DATA = 0x00; // LED off
            delayMs(500);
        }
    }
}
```

```c
// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <6000; j++)
        {
          //do nothing
        }
    }
}
```

Without defining functions

(Example) Input Implementation – LEDs and Switches

Initialization
(Setup registers)

Without defining functions

```c
#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{

  // GPIO Output Initialization (Port F)
  SYSCTL->RCGCGPIO |= 0x20; //Port F enable
  GPIOF->DIR |= 0x0E;      //PF3-1 as output PF4 as input
  GPIOF->AFSEL &= ~(0x0E); //PF3-1 as GPIO controlled by GPIO register
  GPIOF->DEN |= 0x0E; //PF4-1 as digital port
  GPIOF->DATA = 0x11; // initially all 0 output(turn off) and switch on (pull up)

  // GPIO Input Initialization (Port F)
  SYSCTL->RCGCGPIO |= 0x20; //Port F enable
  GPIOF->DIR &= ~(0x10);       //PF0 as input
  GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register
  GPIOF->DEN |= 0x10; //PF4 as digital port
  GPIOF->PUR |= 0x10; //pull up register at PF4 enable

  //Blink LED
  while(1)
  {

    uint8_t ButtonStatus = GPIOF->DATA & 0x10;

    if (ButtonStatus == 0) // switch on (default: pull up)
      {
          GPIOF->DATA = 0x02; // Turn on PF1 (Red)
          delayMs(500);
      }
      else
      {
          GPIOF->DATA = 0x00; // LED off
          delayMs(500);
      }
  }
}
```

```c
// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <6000; j++)
        {
          //do nothing
        }
    }
}
```

(Example) Input Implementation – LEDs and Switches

Without defining functions

infinite loop

Read Switch

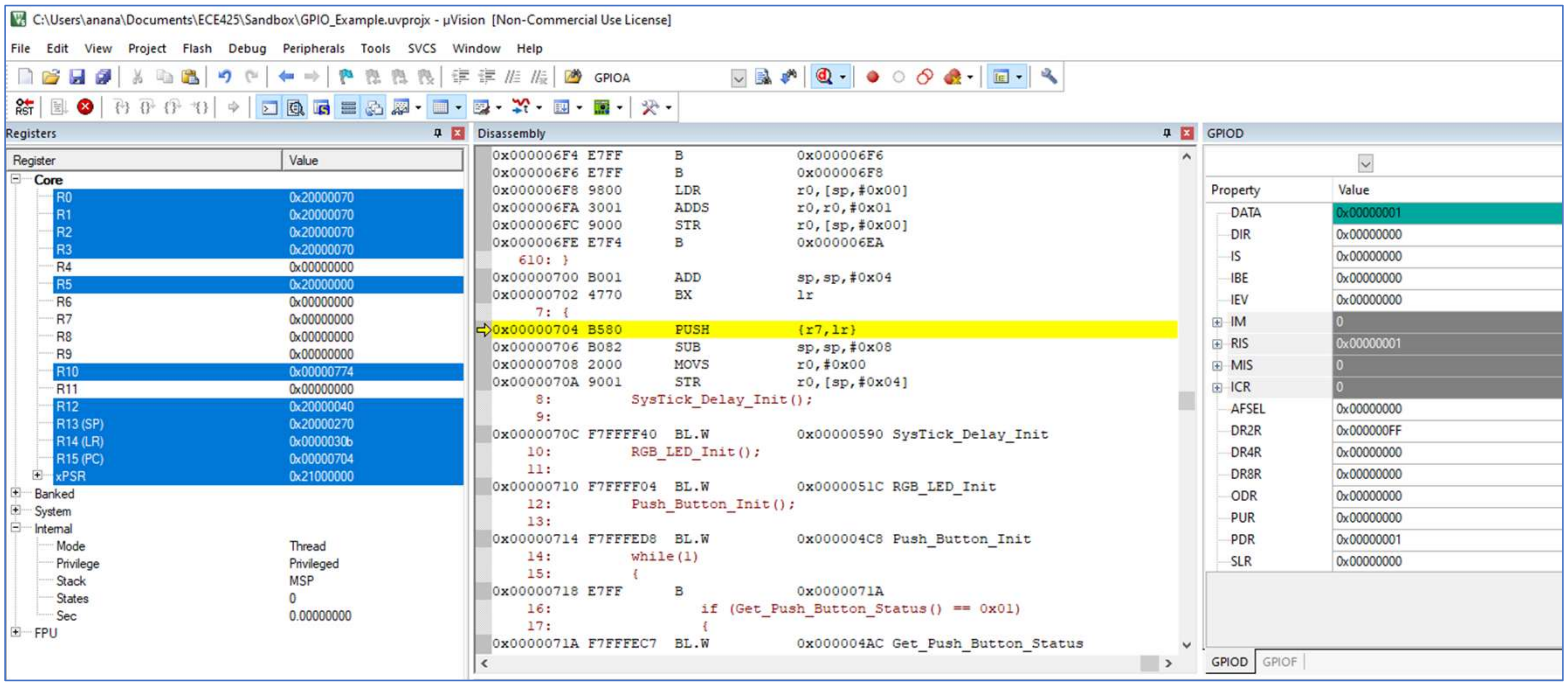
Turn LED on or off (depending on the switch)

```c
#include "TM4C123GH6PM.h"

void delayMs(int n);

int main(void)
{

    // GPIO Output Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
    GPIOF->DIR |= 0x0E;     //PF3-1 as output PF4 as input
    GPIOF->AFSEL &= ~(0x0E); //PF3-1 as GPIO controlled by GPIO register
    GPIOF->DEN |= 0x0E; //PF4-1 as digital port
    GPIOF->DATA = 0x11; // initially all 0 output(turn off) and switch on (pull up)

    // GPIO Input Initialization (Port F)
    SYSCTL->RCGCGPIO |= 0x20; //Port F enable
    GPIOF->DIR &= ~(0x10);      //PF0 as input
    GPIOF->AFSEL &= ~(0x10); //PF4 as GPIO controlled by GPIO register
    GPIOF->DEN |= 0x10; //PF4 as digital port
    GPIOF->PUR |= 0x10; //pull up register at PF4 enable

    //Blink LED
    while(1)
    {

        uint8_t ButtonStatus = GPIOF->DATA & 0x10;

        if (ButtonStatus == 0) // switch on (default: pull up)
        {
            GPIOF->DATA = 0x02; // Turn on PF1 (Red)
            delayMs(500);
        }
        else
        {
            GPIOF->DATA = 0x00; // LED off
            delayMs(500);
        }
    }
}
```

```c
// Delay function
void delayMs(int n)
{
    int i,j;
    for(i=0; i<n; i++)
    {
        for(j=0; j <6000; j++)
        {
          //do nothing
        }
    }
}
```

(Appendix) Input Testing - Debug

- Start a debugging session and then run the program
- Observe the GPIO F registers
  - Peripherals -> System Viewer > GPIO > GPIOF
  - Observe the changes in the DATA registers when SW1 is pressed



Discussion

(Question) Suppose that you need to update your GPIO driver to initialize four push buttons (SW5 to SW2) on the EduBase board instead of only one push button (SW5). What steps do you need to take? Assume that the push buttons use the following pins:

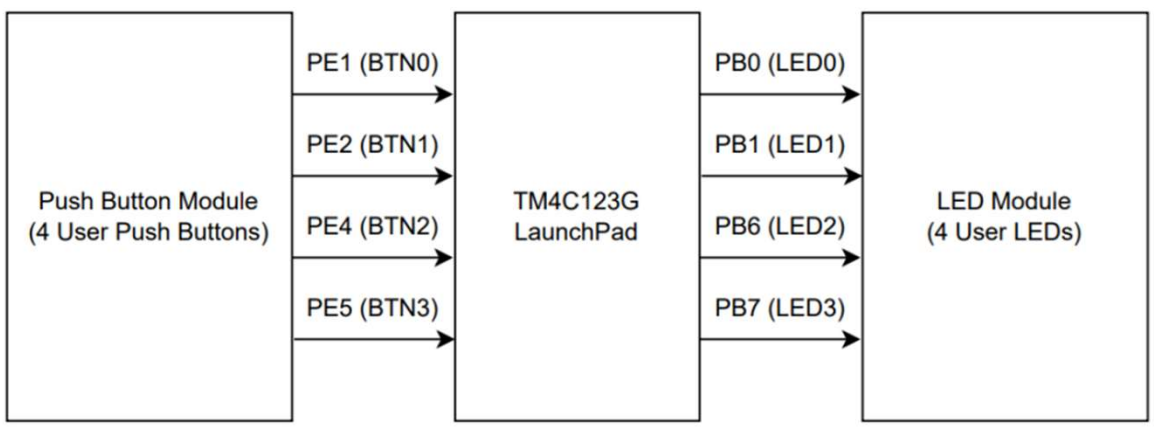
- SW5 - PD0
- SW4 - PD1
- SW3 - PD2
- SW2 - PD3

Appendix

(Appendix) Practice Exam Question

Implement the following functions in C. All functions written must have correct syntax without any errors.

(a) (20 points) Write a *void* function named **BTN_and_LED_Init** that takes no arguments and initializes the following pins as GPIO pins with the specified direction for the system shown in the figure below. You are welcome to write two separate functions (e.g. **BTN_Init** and **LED_Init**) instead of one function. Assume that the push buttons are configured to be active high. Enable the pull-down resistors on the input GPIO pins.

For the output GPIO pins, initialize their values to zero.

- Inputs: PE1, PE2, PE4, PE5
- Outputs: PB0, PB1, PB6, PB7